ENTERPRISE SYSTEM DOCUMENTATION · OFFICIAL TECHNICAL SPECIFICATION

AURA Field Operations Management Platform

Complete Technical Architecture, Data Schemas, APIs, Mobile Screens, and Background GPS Engine Documentation

---

System Name: AURA (Field Officer Management & Tracking Platform)
Architecture Pattern: Offline-First Mobile Client + FastAPI Microservice + Supabase Cloud + React Admin Portal
Client Version: 1.0.0 (Expo SDK 57 / React Native 0.86.3)
Target Platforms: Android (Standalone APK), iOS (Native IPA), Modern Desktop Web (Admin Portal)
Database Engines: SQLite 3 (Client Offline Cache) + PostgreSQL 15 (Supabase Cloud Primary)
Cloud Endpoints: Render (FastAPI Backend: fom-1.onrender.com) · Vercel (Admin Web: fom-kappa.vercel.app)
Classification: Proprietary & Confidential · Parul Chemicals / FSC Dairy

EXECUTIVE OVERVIEW

The AURA platform transforms traditional manual field officer travel tracking into an intelligent, offline-first, BizAnalyst-style mobile and cloud management platform. Field officers execute customer check-ins, record on-site meeting durations, capture up to five visit photographs, and document observations even in complete absence of cellular network connectivity. A low-battery background GPS engine silently logs continuous breadcrumb routes and auto-synchronizes over exponential-backoff pipelines to Supabase and Render. Telecallers and executive fleet administrators gain full visibility through dedicated mobile team dashboards, live Leaflet route mapping, and automated PDF daily travel allowance reports.

1. System Architecture & High-Level Topology

AURA employs a distributed, offline-first multi-tier architecture engineered for extreme reliability in rural and remote agricultural field zones where cellular connectivity is intermittent. The system consists of three distinct deployment tiers:

- **1. Mobile Client Tier (React Native / Expo SDK 57):** Runs on Android and iOS devices. Executes local SQLite transaction queues, offline map rendering, foreground service background GPS breadcrumbing, and multi-photo capture.
- **2. Cloud Services Tier (FastAPI on Render):** Python 3.11 asynchronous microservice exposing 37 REST API endpoints for user profile assignment, customer master synchronization, GPS batch ingestion, and Supabase Storage photo bridging.
- **3. Cloud Data Tier (Supabase PostgreSQL + Storage):** Hosted database providing Row-Level Security (RLS), real-time JWT authentication, persistent storage for visit photos in the visit-images bucket, and direct REST fallback ingestion.
- **4. Web Admin Tier (React 18 + Vite on Vercel):** Single Page Application (SPA) offering 4 operational consoles: Real-time Visit Timeline, GPS Route Mapping on Leaflet vector tiles, Daily Travel & Allowance Calculator, and Caller-to-Officer Assignment Portal.

TIER	COMPONENT	RESPONSIBILITY & TECH STACK
Mobile Client	AURA Mobile App (Android/iOS)	Expo SDK 57, React Native 0.86, SQLite (WAL), Zustand Store, TaskManager background GPS, expo-print PDF generator.
API Gateway	FastAPI Service (Render Cloud)	Python 3.11, Pydantic v2 schemas, JWT auth verification, multipart photo streaming, health checks, route telemetry.
Database & Auth	Supabase Cloud (PostgreSQL 15)	PostgreSQL tables with RLS policies, GoTrue Auth engine, Postgres triggers (handle_new_user), S3-compatible Storage (visit-images).
Direct REST Fallback	Supabase PostgREST	Instant 100-200ms batch ingestion bypassing Render backend cold starts during network reconnection.
Web Console	Admin Portal (Vercel Cloud)	React 18, Vite, Tailwind CSS, Leaflet.js interactive maps, jsPDF + autotable report export engine.

2. Role-Based Access Control (RBAC) System

AURA implements a strict 3-tier organizational hierarchy. The user role is stored as the single source of truth in the Supabase profiles.role column and encoded in the JWT claims:

ROLE	TARGET INTERFACE	PERMITTED CAPABILITIES & WORKFLOWS
field_officer (Field Officer)	Mobile App (FO Workspace)	• Navigation: **Home, Customers, History, Settings**. • Silent background GPS tracking (Track/Map tabs hidden). • Customer Check-In, Live elapsed timer, Notes, Camera capture (1-5 photos). • Check-Out calculation, local SQLite persistence, automatic cloud sync. • Customer master additions with offline synchronization.

caller (Telecaller)	Mobile App (Caller Workspace)	• Navigation: **Team, Reports, Settings**. • Supervised view of all assigned Field Officers (permanent assignment). • Daily visit counts, total customer meeting time, individual officer breakdowns. • Mobile PDF visit report generation and sharing via native share sheet. • Cannot execute check-ins or log personal GPS tracking.
admin (Fleet Executive)	Web Admin Portal (Desktop Web)	• Navigation: **Visit Dashboard, GPS Map, Daily Report, Team Management**. • Fleet-wide visit timeline with notes and attached photo viewer. • Interactive Leaflet map with breadcrumb route lines, start/end pins, telemetry. • Daily travel mileage calculation, customizable allowance rules, CSV/PDF export. • Full assignment control: assign/reassign FOs to Callers; promote/demote roles.

CONFIDENTIAL · PARUL CHEMICALS / FSC DAIRY · ALL RIGHTS RESERVED

3. Mobile Application Modules & Screen Specifications

The mobile client is built on Expo SDK 57 with Expo Router file-based navigation. Tab bars and accessible screens dynamically adapt based on the authenticated user's profile role.

SCREEN ROUTE	ACCESSIBILITY	UI/UX BEHAVIOR & ARCHITECTURAL IMPLEMENTATION
app/login.tsx	Public / Unauthenticated	Branded AURA symbol header, email/password inputs, toggle password visibility, 'Keep me signed in' checkbox. Authenticates via Supabase Auth (primary), FastAPI /auth/login (secondary), or designated officer roster (offline fallback). Persists session to SQLite app_settings.
app/(tabs)/index.tsx (FO Home)	Field Officer	Greets officer by first name with date header. Shows active visit status banner with pulsing green dot and live elapsed timer. Prominent **CHECK IN NOW** button. Metric cards for 'Visits Today' and 'Total Time'. Scrollable list of today's completed visits with customer name, times, notes, and duration badges.
app/checkin/select-customer.tsx	Field Officer	Instant search input filtering local SQLite customer master list by name, village, or taluka. Displays 'Add New Customer' quick-action button. Selecting a customer fetches instant GPS coordinates (Location.Accuracy.Balanced) and transitions to active visit screen.
app/checkin/active.tsx (Active Check-in)	Field Officer	Customer pin icon and name. Prominent blue timer card displaying live wall-clock elapsed time (HH:MM:SS) with LIVE pill. Text area for visit notes. Photo attachment section supporting up to 5 photos via camera or library (expo-image-picker). High-contrast red **CHECK OUT** button with confirmation alert.
app/checkin/add-customer.tsx	Field Officer	Form inputs for Customer Name (required), Village, Taluka, District, Pincode, and Contact Phone. Saves record locally with sync_status='pending' and auto-selects newly created customer for immediate check-in.
app/(tabs)/customers.tsx	Field Officer	Comprehensive customer directory with real-time search. Displays customer initials avatar, business name, and village/taluka badges. Supports instant offline lookup from SQLite.
app/(tabs)/history.tsx (Visit History)	Field Officer / Caller	Replaced legacy session view with full visit audit log. Groups past visits into day-wise section headers (newest day first). Cards show customer name, Done/In-Progress badge, check-in to check-out times, duration pill, observation notes, photo thumbnails, and sync indicator icon (synced/pending/failed).
app/(tabs)/team.tsx (Caller Dashboard)	Caller	Supervisor dashboard listing all assigned field officers. Summary cards: Total Team Visits, Combined Field Time, Active Officers. Date filter pills (Today, Yesterday, Custom Date). Officer cards breakdown meetings and time. 'Download PDF Report' button generates high-res PDF via expo-print and invokes native share sheet.
app/(tabs)/settings.tsx	All Authenticated	Displays user name, email, role badge, assigned caller (for FO). Battery optimization profile selector (Ultra, Balanced, High). Offline database sync queue count with manual 'Sync Now' trigger. Offline map cache management (300km vector tile pre-download). Secure Logout button.
app/(tabs)/map.tsx	Admin / Testing (Hidden for FO)	MapLibre GL native vector tile map rendering OpenFreeMap Liberty style. GPS track recording polyline, start/stop pins, real-time speed, altitude, accuracy telemetry HUD.

CONFIDENTIAL · PARUL CHEMICALS / FSC DAIRY · ALL RIGHTS RESERVED

4. Background GPS Engine & Battery Duty Cycling

Continuous GPS tracking on modern mobile operating systems (especially Android 12+) presents severe challenges due to aggressive battery optimization (Doze mode), background execution limits, and OEM task killers (Samsung, Xiaomi, OnePlus, Vivo). AURA implements a robust, hardware-level architecture to ensure uninterrupted tracking:

- **Module-Scope Task Definition:** TaskManager.defineTask(BACKGROUND_LOCATION_TASK, ...) is registered in global scope outside the React component tree in locationService.ts. This enables Android's Headless JS runtime to deliver batched coordinates even when the app activity is suspended or destroyed.
- **Persistent Foreground Service:** Invoked with killServiceOnDestroy: false, high-priority notification channel, and continuous lock screen indicator. Android cannot terminate the tracking process without explicit user action.
- **High Accuracy Requirement:** Android FusedLocationProvider restricts Location.Accuracy.Balanced to Wi-Fi/Cell towers, which power down when the screen turns off. AURA enforces Location.Accuracy.High (PRIORITY_HIGH_ACCURACY) to keep the GPS chip bound to the foreground notification.
- **Hardware Duty Cycling:** Instead of burning battery with continuous 1-second streaming, AURA requests GPS fixes every 15 seconds (balanced mode) or 30 seconds (ultra mode), allowing the GPS RF receiver to enter low-power sleep states between fixes (~3-4% battery drain per hour).
- **Zero Deferred Updates Lag:** Sets deferredUpdatesInterval: 0 and deferredUpdatesDistance: 0 so Android delivers fixes immediately without waiting for the officer to move arbitrary distances.
- **Kalman Filtering & Anti-Drift:** defaultKalmanFilter.ts applies a 2D coordinate covariance matrix to de-noise raw fixes. Points with accuracy error > 75m or stationary drift moves < 1.5m at speeds < 0.6 m/s are automatically suppressed.
- **Wall-Clock Synchronization:** Active visit timers in visitStore.ts never count setInterval ticks. They compute Math.floor((Date.now() - checkinMs) / 1000) against system wall-clock time, ensuring zero drift when waking up from lock screen.

PROFILE	INTERVAL	DISTANCE	ACCURACY	BATTERY IMPACT	TARGET USE CASE
ultra Ultra Saver	30,000 ms (30s)	15 meters	High (GPS)	~2% / hr (16+ hours)	Full-day 14+ hr field shifts with heavy stationary customer meetings.
balanced (Default)	15,000 ms (15s)	10 meters	High (GPS)	~3-4% / hr (12+ hours)	Standard field officer operations balancing curve accuracy with battery.
high Precision	8,000 ms (8s)	5 meters	High (GPS)	~7-9% / hr (6-8 hours)	Urban route auditing, rapid vehicle travel, high-precision turn analysis.

5. Data Schemas: Supabase PostgreSQL & Local SQLite

AURA maintains an identical conceptual schema across Supabase Cloud (PostgreSQL 15) and mobile clients (SQLite 3). This ensures seamless two-way synchronization and offline transaction safety.

5.1 Supabase Cloud Tables (PostgreSQL 15)

TABLE	COLUMN	TYPE	CONSTRAINTS & DESCRIPTION
profiles	id	UUID (PK)	FK to auth.users.id (CASCADE).
	full_name	TEXT	Full name of employee.
	role	TEXT	CHECK (role IN ('admin', 'caller', 'field_officer')).
	caller_id	UUID (FK)	Self-FK to profiles.id (assigned telecaller).
	created_at	TIMESTAMPTZ	Record creation timestamp.
customers	id	UUID (PK)	Default gen_random_uuid().
	name	TEXT	Business / Farmer / Account name (NOT NULL).
	village	TEXT	Village / Settlement name.
	taluka	TEXT	Sub-district (Taluka).
	district	TEXT	District.
	pincode	TEXT	Postal pincode.
	contact_no	TEXT	Phone contact number.
	created_by	UUID (FK)	FK to auth.users.id (creator officer).
	created_at	TIMESTAMPTZ	Record creation timestamp.
	is_active	BOOLEAN	Soft deletion flag (default true).
customer_visits	id	UUID (PK)	Default gen_random_uuid().
	user_id	UUID (FK)	FK to auth.users.id (Field Officer).
	customer_id	UUID (FK)	FK to customers.id.
	checked_in_at	TIMESTAMPTZ	Check-in timestamp (NOT NULL).
	checked_out_at	TIMESTAMPTZ	Check-out timestamp (NULL during active visit).
	duration_minutes	INT	Computed on-site duration in minutes.
	checkin_lat	FLOAT	WGS-84 check-in latitude.
	checkin_lng	FLOAT	WGS-84 check-in longitude.
	notes	TEXT	Field notes / meeting observations.
	photo_urls	JSONB	JSON array of public Supabase Storage photo URLs.
	sync_status	TEXT	Sync tracking state ('synced', 'pending').
	created_at	TIMESTAMPTZ	Record timestamp.
tracking_sessions	id	TEXT (PK)	Session identifier (e.g. visit-UUID).
	user_id	UUID (FK)	FK to auth.users.id.
	started_at	TIMESTAMPTZ	Trip start time.
	ended_at	TIMESTAMPTZ	Trip completion time.
	total_distance_km	FLOAT	Cumulative GPS distance in kilometers.
	status	TEXT	'active', 'paused', 'completed'.
location_points	id	TEXT (PK)	Point UUID.
	session_id	TEXT (FK)	FK to tracking_sessions.id.
	user_id	UUID (FK)	FK to auth.users.id.
	latitude	FLOAT	Kalman-filtered latitude.
	longitude	FLOAT	Kalman-filtered longitude.
	recorded_at	TIMESTAMPTZ	GPS fix timestamp.
	accuracy_m	FLOAT	GPS horizontal accuracy in meters.
	altitude_m	FLOAT	Altitude in meters above sea level.
	speed_mps	FLOAT	Velocity in meters per second.
	synced_at	TIMESTAMPTZ	Server ingestion timestamp.

CONFIDENTIAL · PARUL CHEMICALS / FSC DAIRY · ALL RIGHTS RESERVED

5.2 Mobile Local SQLite Tables (gps_tracker.db)

SQLITE TABLE	SCHEMA DEFINITION & PURPOSE
app_settings	(key TEXT PRIMARY KEY, value TEXT NOT NULL) Stores persistent key-value configuration: auth_session (logged-in user credentials and JWT token), active_session_id (active tracking ID surviving app restarts), battery_mode ('ultra'|'balanced'|'high').
customers	(id TEXT PK, name TEXT, village TEXT, taluka TEXT, district TEXT, pincode TEXT, contact_no TEXT, created_at TEXT, sync_status TEXT) Full offline customer master. Supports instant client-side LIKE queries during search.
customer_visits	(id TEXT PK, customer_id TEXT, customer_name TEXT, checked_in_at TEXT, checked_out_at TEXT, duration_minutes INTEGER, checkin_lat REAL, checkin_lng REAL, notes TEXT, photo_uris TEXT, sync_status TEXT) Offline visit records. photo_uris holds local JSON string array of file:// paths before cloud upload.
tracking_sessions	(id TEXT PK, started_at TEXT, ended_at TEXT, total_distance_km REAL, status TEXT, point_count INTEGER, sync_status TEXT) Tracks individual travel routes and check-in background GPS sessions.
location_points	(id TEXT PK, session_id TEXT, latitude REAL, longitude REAL, recorded_at TEXT, accuracy_m REAL, altitude_m REAL, speed_mps REAL, sync_status TEXT) High-volume breadcrumb points. Status transitions atomically from 'pending' -> 'in_flight' -> 'synced'.

CONFIDENTIAL · PARUL CHEMICALS / FSC DAIRY · ALL RIGHTS RESERVED

6. Offline-First Synchronization Architecture

AURA's synchronization engine is designed with zero-data-loss guarantees. Data is ALWAYS written to local SQLite first with an initial status of pending. The upload engine operates autonomously in the background via two parallel pipelines:

- **Two-Tier Resilient Ingestion:** When uploading breadcrumb points, the app attempts the FastAPI Render backend first. If Render is asleep (cold starting), it instantly falls back to direct Supabase PostgREST ingestion in 100-200ms. Field officers never experience lag.
- **Atomic Transaction Locking:** claimPendingPoints(batchSize) executes inside an SQLite transaction, transitioning records to in_flight. This strictly prevents race conditions between network reconnection events and timer-based auto-sync cycles.
- **Exponential Backoff & Failure Recovery:** If an upload fails due to packet drops, points are released back to pending. resetFailedPoints() automatically cleans stale in-flight claims upon next retry.
- **Visit & Photo Upload Pipeline:** visitSyncService.ts synchronizes completed visits through a 3-step sequence: (1) Push Check-In payload, (2) Multipart upload of photos to /api/v1/visits/{id}/photos which streams images to Supabase Storage visit-images bucket, (3) Push Check-Out payload containing returned public CDN URLs.
- **Continuous Auto-Sync Trigger:** Listens to NetInfo network changes (offline -> online transition) plus a continuous 25-second periodic pulse while online.

STAGE	TRIGGER	OPERATION & FAILSAFE BEHAVIOR
1. Offline Capture	Officer Check-In / Check-Out	Inserts into SQLite customer_visits and tracking_sessions with sync_status='pending'. Photos saved to app's document directory.
2. Connectivity Detection	NetInfo event / 25s timer	Evaluates network quality via lightweight health ping. If online and not currently sync-locked, initiates upload.
3. Atomic Point Claim	Sync engine worker	Atomically marks up to 50 points as in_flight within SQLite transaction to prevent duplicate uploads.
4. Photo Streaming	visitSyncService	Uses FileSystem.uploadAsync with multipart/form-data. Backend uploads to Supabase Storage bucket visit-images.
5. Commit & Clean	HTTP 200/201 response	Updates local SQLite status to synced. Upon network interruption, resets in-flight records to pending.

7. Backend API Specification (FastAPI / Render)

The backend service is built with FastAPI and deployed on Render. It exposes 37 endpoints categorized across 7 routers. All endpoints under /api/v1 enforce Bearer JWT authentication (unless designated as public auth or health check).

METHOD	PATH	AUTH	DESCRIPTION & PAYLOAD
GET	`/api/v1/customers`	Authenticated	**Retrieve master customer list for offline caching** Req: None Res: List[CustomerResponse]
POST	`/api/v1/customers`	Authenticated	**Create a new customer from mobile app** Req: {name, village, taluka, district, pincode, contact_no} Res: CustomerResponse (201)
GET	`/api/v1/customers/{id}`	Authenticated	**Get customer details by UUID** Req: Path: id (UUID) Res: CustomerResponse (200)
PUT	`/api/v1/customers/{id}`	Authenticated	**Update customer contact or location** Req: {name, village, taluka, district, pincode, contact_no} Res: CustomerResponse (200)
POST	`/api/v1/visits/checkin`	FO / Caller / Admin	**Record field customer check-in** Req: {visit_id, customer_id, checkin_lat, checkin_lng, checked_in_at} Res: {status: 'checked_in', visit_id} (201)
POST	`/api/v1/visits/checkout`	FO / Caller / Admin	**Record check-out, duration, and photos** Req: {visit_id, notes, checked_out_at, duration_minutes, photo_urls[]} Res: {status: 'checked_out', duration_minutes} (200)
POST	`/api/v1/visits/{visit_id}/photos`	FO / Caller / Admin	**Upload visit photo to Supabase Storage** Req: Multipart: file (image/jpeg, png) Res: {status: 'uploaded', public_url, path} (201)
GET	`/api/v1/visits/today`	Field Officer	**Get officer's completed and active visits today** Req: Query: ?target_date=YYYY-MM-DD Res: List[VisitResponse] (200)
GET	`/api/v1/visits/history`	Field Officer	**Get officer's past visits (paginated)** Req: Query: ?limit=50&offset;=0 Res: List[VisitResponse] (200)
GET	`/api/v1/visits/officer/{user_id}`	Caller / Admin	**Get visits of specific officer** Req: Path: user_id (UUID), Query: ?target_date Res: List[VisitResponse] (200)
GET	`/api/v1/visits/summary`	Caller / Admin	**Team visit totals and metrics** Req: Query: ?start_date=&end;_date= Res: {total_visits, total_hours, officers_count} (200)
GET	`/api/v1/visits/admin/timeline`	Admin / Caller	**Aggregated officer visit timeline with notes** Req: Query: ?target_date=YYYY-MM-DD Res: List[OfficerTimelineResponse] (200)
GET	`/api/v1/profiles/me`	Authenticated	**Get current user profile, role, and caller info** Req: None Res: ProfileResponse (200)
GET	`/api/v1/profiles/team`	Caller	**Get caller's assigned field officers and visit stats** Req: Query: ?target_date=YYYY-MM-DD Res: List[TeamOfficerResponse] (200)

GET	/api/v1/admin/callers	Admin	**List all registered telecallers** Req: None Res: List[CallerResponse] (200)
GET	/api/v1/admin/field-officers	Admin	**List all field officers and assigned callers** Req: None Res: List[FieldOfficerResponse] (200)
GET	/api/v1/admin/callers/{id}/officers	Admin	**List officers assigned to specific caller** Req: Path: caller_id (UUID) Res: List[FieldOfficerResponse] (200)
POST	/api/v1/admin/profiles/assign	Admin	**Permanently assign FO to Caller** Req: {officer_id: UUID, caller_id: UUID} Res: {status: 'assigned', officer_id, caller_id} (200)
PUT	/api/v1/admin/profiles/{id}/role	Admin	**Update user role (admin/caller/field_officer)** Req: {role: 'admin'|'caller'|'field_officer'} Res: {status: 'updated', user_id, role} (200)
GET	/api/v1/admin/all-users	Admin	**List all users across organization** Req: None Res: List[UserProfileResponse] (200)
GET	/api/v1/admin/overview	Admin	**High-level fleet summary counts** Req: None Res: {active_officers, total_km, total_visits} (200)
GET	/api/v1/admin/officers	Admin	**Fleet officer GPS tracking status roster** Req: None Res: List[OfficerStatusResponse] (200)
GET	/api/v1/admin/officers/{id}/sessions	Admin	**GPS sessions for specific officer** Req: Path: user_id (UUID), Query: ?date= Res: List[SessionResponse] (200)
GET	/api/v1/admin/sessions/{id}/route	Admin	**High-res GPS route coordinates for map** Req: Path: session_id (TEXT) Res: {session, points: List[PointResponse]} (200)
POST	/api/v1/sessions	Authenticated	**Initialize GPS tracking session** Req: {session_id, started_at} Res: {status: 'created', session_id} (201)
GET	/api/v1/sessions	Authenticated	**List user's GPS tracking sessions** Req: Query: ?limit=20 Res: List[SessionResponse] (200)
GET	/api/v1/sessions/{id}	Authenticated	**Get GPS session details** Req: Path: session_id Res: SessionResponse (200)
POST	/api/v1/sessions/{id}/finish	Authenticated	**Complete GPS tracking session** Req: {ended_at, total_distance_km} Res: {status: 'finished', total_distance_km} (200)
POST	/api/v1/sessions/{id}/locations/batch	Authenticated	**Batch upload GPS coordinates (up to 50)** Req: {points: [{id, lat, lon, recorded_at, accuracy_m, speed_mps}]} Res: {status: 'ingested', count} (200)
GET	/api/v1/sessions/{id}/locations	Authenticated	**Fetch coordinates for a session** Req: Path: session_id Res: List[LocationPointResponse] (200)

POST	/api/v1/auth/login	Public	**Backend credential login fallback** Req: {email, password} Res: {access_token, token_type, user_id, role} (200)
POST	/api/v1/auth/create-user	Admin	**Admin user creation endpoint** Req: {email, password, full_name, role} Res: {user_id, email, role} (201)
GET	/health	Public	**Microservice health check & keep-alive** Req: None Res: {status: 'ok'} (200)

8. Admin Web Portal Consoles & Analytics

The Admin Web Portal (deployed at https://fom-kappa.vercel.app) is built with React 18, Vite, and Tailwind CSS. It provides fleet managers with complete visibility and operational governance across 4 core tabs:

Tab 1: Visit Timeline Dashboard

Consumes /api/v1/visits/admin/timeline?target_date=YYYY-MM-DD. Groups visits by field officer. Displays each officer's total customer meetings, total on-site time, and chronologically ordered meeting cards. Each meeting card displays customer name, village/taluka badge, check-in to check-out timestamps, calculated duration, officer observation notes, and expandable thumbnails of captured field photographs.

Tab 2: Interactive GPS Map & Sessions

Consumes /api/v1/admin/officers and /api/v1/admin/sessions/{id}/route. Renders high-speed Leaflet.js vector tiles. Displays officer status (Active/Idle), daily cumulative distance, and selectable session dropdowns. Renders GPS route polylines with green start markers, checkered finish markers, and interactive popups with speed and timestamp telemetry.

Tab 3: Daily Travel & Allowance Report

Calculates field allowances based on cumulative travel kilometers. Features a date-range selector, search filtering, and dynamic allowance tier calculation. Includes one-click **Export CSV** for accounting software and **Export PDF** generating clean tabular summaries via jspdf + jspdf-autotable.

Tab 4: Team Management & Role Administration

Consumes /api/v1/admin/callers, /api/v1/admin/field-officers, and /api/v1/admin/all-users. Allows administrators to permanently assign Field Officers to Telecallers with instant UI updates via POST /api/v1/admin/profiles/assign. Enables role upgrades and downgrades ('admin', 'caller', 'field_officer') via PUT /api/v1/admin/profiles/{id}/role.

CONFIDENTIAL · PARUL CHEMICALS / FSC DAIRY · ALL RIGHTS RESERVED

9. PDF Generation Engines (Mobile & Web)

AURA incorporates two dedicated client-side PDF generation engines tailored for their respective operating environments:

9.1 Mobile PDF Engine (expo-print + expo-sharing)

Implemented in mobile/src/services/pdfReportService.ts. Engineered specifically for Telecallers on mobile devices. Generates a complete daily team visit audit report entirely on-device without server dependency.

Key Features:
• Resolves local photo file URIs (file://...) into base64 data URIs so captured photos embed cleanly offline.
• Generates HTML markup with responsive print media CSS (page-break-inside avoid, clean typography, metric cards).
• Invokes Print.printToFileAsync({ html }) to render the native OS print engine to a temporary PDF file.
• Invokes Sharing.shareAsync(uri) allowing the caller to send the PDF via WhatsApp, Email, or Google Drive.

9.2 Web Admin PDF Engine (jsPDF + jspdf-autotable)

Implemented in admin-portal/src/utils/pdfExport.ts. Engineered for desktop web browsers with zero external print dialogs.

Key Features:
• Vector-quality PDF rendering using jsPDF coordinate canvas.
• Automatically formats officer name, meeting count, total duration, customer lists, and notes into structured tabular layouts.
• Applies corporate color palettes, striped table rows, and page numbering.
• Triggers immediate browser file download (e.g. AURA_Visit_Report_2026-10-31.pdf).

CONFIDENTIAL · PARUL CHEMICALS / FSC DAIRY · ALL RIGHTS RESERVED

10. Deployment & Build Infrastructure

The AURA ecosystem is deployed across automated, resilient cloud infrastructures:

TIER	PROVIDER & URL	CONFIGURATION & ENVIRONMENT DETAILS
Mobile App (APK)	Expo Application Services (EAS)	EAS project ID: 97882ef8-4f3c-4127-9aee-27617234bcf4. Profile: preview (standalone Android APK). Embedded env vars: EXPO_PUBLIC_API_URL, EXPO_PUBLIC_SUPABASE_URL, EXPO_PUBLIC_SUPABASE_ANON_KEY, EXPO_PUBLIC_MAP_STYLE_URL.
Backend API	Render Cloud https://fom-1.onrender.com	Python 3.11 Web Service running Uvicorn ASGI server. CORS allowed origins configured for mobile schemes and Vercel domains. Connected to Supabase PostgreSQL via connection pooling.
Admin Portal	Vercel Cloud https://fom-kappa.vercel.app	Vite static build pipeline. vercel.json rewrites routing all traffic to /index.html for client-side React Router navigation.
Cloud Database	Supabase https://erggjpdfkfaxpnxpieho.supabase.co	PostgreSQL 15 database instance. Public storage bucket: visit-images. JWT authentication provider with custom user metadata claims.
Source Control	GitHub infotechparulchemicals1-beep/fom	Main production git repository. Committer identity: devscode1 / 23ce137@svitvasad.ac.in.

DOCUMENT VALIDATION & RELEASE APPROVAL

This technical specification accurately documents the complete architecture, data models, mobile modules, background GPS engine, and cloud endpoints of the AURA platform as implemented.
Prepared For: Parul Chemicals / FSC Dairy Operations Team
Document Status: RELEASED & VERIFIED · V1.0.0